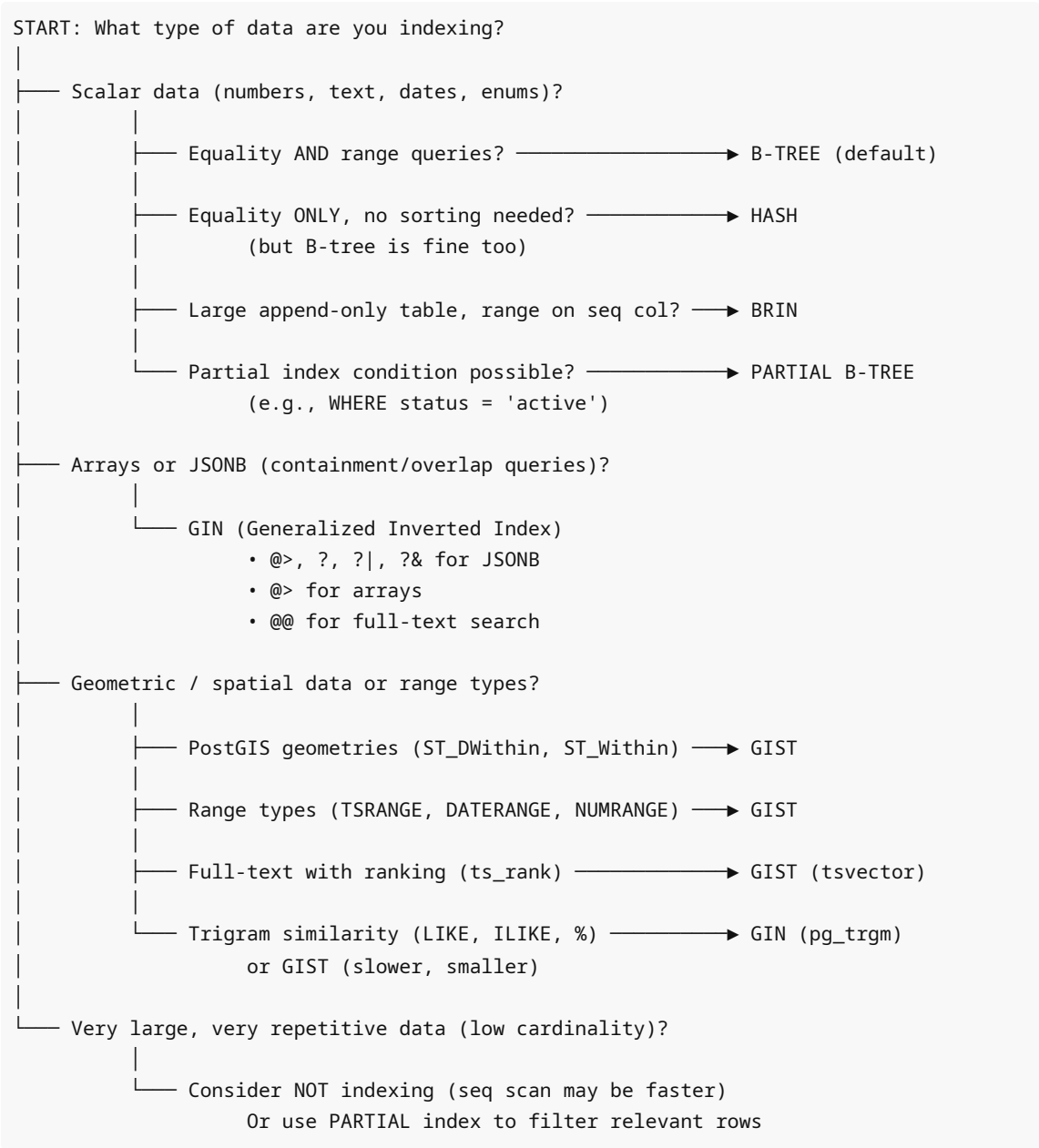


Index Decision Cheat Sheet

Decision tree + tables for choosing the right PostgreSQL index type.

Decision Tree (ASCII)



Index Types Reference Table

Index Type	Best For	Operators	Write Cost	Index Size	Notes
------------	----------	-----------	------------	------------	-------

B-Tree	Most scalar data	=, <, >, <=, >=, BETWEEN, IN, LIKE 'foo%'	Low	Medium	Default; supports sorting
Hash	Exact equality only	= only	Low	Smallest	Faster equality than B-tree in theory; NOT replicated pre-PG10
GIN	Arrays, JSONB, FTS	@>, ?, `?	, ?&, @@, &&`	High	Largest
GiST	Geometry, ranges, FTS	@>, <@, &&, <-> (kNN), @@	Medium	Medium	Lossy (rechecks needed), supports ordering
SP-GiST	Hierarchical, non-balanced	=, <, >, <@, @>	Medium	Small	Good for IP addresses, phone numbers, quadrees
BRIN	Huge append-only tables	=, <, >, range	Very Low	Tiny (~1/1000 of B-tree)	Block-level ranges; only useful if data is physically correlated with column

When to Use Each Type

B-Tree (Use for ~90% of indexes)

```
-- Standard: equality + range
CREATE INDEX idx_orders_date ON orders (created_at);
CREATE INDEX idx_users_email ON users (email);

-- Composite: most common filter first
CREATE INDEX idx_orders_customer_date ON orders (customer_id, created_at DESC);

-- With included columns (covering index)
CREATE INDEX idx_orders_covering ON orders (customer_id, status) INCLUDE
(total_amount);
-- Query: SELECT total_amount WHERE customer_id=X AND status='active' - index-only
scan!

-- Descending order
CREATE INDEX idx_recent_first ON orders (created_at DESC);

-- LIKE prefix match (only works with LIKE 'prefix%', not '%suffix')
CREATE INDEX idx_names ON users (name text_pattern_ops); -- for C locale
```

Hash

```
-- Exact equality only, never range queries
CREATE INDEX idx_session_hash ON sessions USING HASH (session_token);
-- session_token = 'abc123' → very fast
-- session_token > 'abc123' → NOT supported, falls back to seq scan
```

GOTCHA: B-Tree index is usually equally fast for equality. Hash saves space but loses range capability. Only use Hash when you're 100% sure no range queries will ever run.

GIN (Generalized Inverted Index)

```
-- Arrays
CREATE INDEX idx_tags ON posts USING GIN (tags);
-- Query: WHERE 'postgresql' = ANY(tags) or WHERE tags @> ARRAY['postgresql']

-- JSONB
CREATE INDEX idx_metadata ON events USING GIN (metadata);
-- Query: WHERE metadata ? 'user_id' or WHERE metadata @> '{"status": "active"}'

-- JSONB specific path (faster, smaller)
CREATE INDEX idx_status ON events ((metadata->>'status')); -- B-tree on extracted field

-- Full-text search
CREATE INDEX idx_fts ON articles USING GIN (to_tsvector('english', title || ' ' || body));

-- Trigram (requires pg_trgm extension)
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_trgm ON products USING GIN (name gin_trgm_ops);
-- Supports: LIKE '%middle%', ILIKE, similarity()
```

GOTCHA: GIN is expensive to write (maintains inverted index for every element). For JSONB, always consider indexing extracted fields with B-tree if the field has low cardinality.

GiST (Generalized Search Tree)

```
-- PostGIS geometry
CREATE EXTENSION postgis;
CREATE INDEX idx_geo ON locations USING GIST (location);
-- Supports: ST_DWithin, ST_Within, ST_Intersects, ST_Contains
-- Operator: location && bbox, location <-> point (kNN ordering)

-- Range types
CREATE EXTENSION btree_gist; -- needed for some range operators
CREATE INDEX idx_schedules ON events USING GIST (during);
-- Supports: during && other_range, during @> timestamp, etc.

-- Exclusion constraints (use GIST)
CREATE TABLE reservations (
```

```

    room_id INT,
    during TSRANGE,
    EXCLUDE USING GIST (room_id WITH =, during WITH &&) -- no overlapping
reservations
);

-- Full-text with tsvector + ranking
CREATE INDEX idx_fts_gist ON articles USING GIST (to_tsvector('english', body));
-- GiST is smaller than GIN for FTS but slower for most workloads

-- Trigram (alternative to GIN, smaller but slower)
CREATE INDEX idx_trgm_gist ON products USING GIST (name gist_trgm_ops);

```

BRIN (Block Range INdex)

```

-- Huge tables with naturally correlated data (inserts in timestamp order)
CREATE INDEX idx_events_brin ON events USING BRIN (occurred_at);
-- Only ~128 pages to scan for time range queries on 10B row table!

-- Custom range size (default: 128 pages)
CREATE INDEX idx_logs_brin ON logs USING BRIN (created_at) WITH (pages_per_range =
64);

-- Works for any naturally ordered column:
CREATE INDEX idx_users_brin ON users USING BRIN (user_id); -- auto-increment

```

GOTCHA: BRIN is *ONLY* useful when data is physically ordered by the column. A BRIN on `email` (random order) is useless — it will never prune blocks effectively.

Partial Indexes — Big Wins

```

-- Only active records (90% of queries hit WHERE is_active = TRUE)
CREATE INDEX idx_users_active ON users (email) WHERE is_active = TRUE;

-- Only unprocessed jobs
CREATE INDEX idx_jobs_pending ON jobs (created_at, priority DESC) WHERE status =
'pending';

-- Only non-null values
CREATE INDEX idx_phone ON users (phone) WHERE phone IS NOT NULL;

-- Only recent data (hot path)
CREATE INDEX idx_recent_orders ON orders (customer_id, created_at DESC)
WHERE created_at >= '2024-01-01';
-- Won't work for older queries, but saves space and improves inserts on current
data

```

Composite Index Rules

```
-- Column order matters: most selective first, most frequently filtered first
-- For query: WHERE a = 1 AND b = 2 AND c = 3
CREATE INDEX idx ON t (a, b, c);    -- ✓ all 3 columns used
CREATE INDEX idx ON t (c, b, a);    -- ✗ only uses c for exact match first

-- Range predicate: range column LAST in composite
-- For query: WHERE status = 'active' AND created_at >= '2024-01-01'
CREATE INDEX idx ON t (status, created_at); -- ✓ both columns used
CREATE INDEX idx ON t (created_at, status); -- ✗ only created_at range used

-- Covering index: INCLUDE non-indexed columns (PostgreSQL 11+)
CREATE INDEX idx_orders ON orders (customer_id, status)
INCLUDE (total_amount, created_at);
-- Allows index-only scan for: SELECT total_amount, created_at WHERE customer_id=X
AND status='Y'
```

Index Gotchas

Gotcha	Detail	Fix
Function on indexed column	WHERE UPPER(email) = ... ignores index on email	Create index on UPPER(email) or use citext type
Implicit type cast	WHERE int_col = '42' (string) ignores int index	Cast correctly or fix application
OR condition	WHERE a = 1 OR b = 2 may not use index on a	Use UNION ALL or ensure both columns indexed
IS NULL	B-tree stores NULLs, so IS NULL can use index	Fine, but verify with EXPLAIN
LIKE '%pattern%'	Leading wildcard can't use B-tree	Use GIN + pg_trgm
Unused indexes	Index exists but 0 idx_scan	Check pg_stat_user_indexes; consider dropping
Index on low-cardinality column	e.g., boolean, status (3 values)	Partial index or skip indexing
Bloated indexes	After many deletes/updates	REINDEX INDEX idx; or VACUUM

Functional / Expression Indexes

```
-- Case-insensitive unique email
CREATE UNIQUE INDEX idx_email_ci ON users (LOWER(email));
-- Query must also use LOWER: WHERE LOWER(email) = LOWER(:input)
```

```

-- Date part
CREATE INDEX idx_birth_year ON users (EXTRACT(YEAR FROM birth_date));
-- Query: WHERE EXTRACT(YEAR FROM birth_date) = 1990

-- JSONB field extraction
CREATE INDEX idx_user_type ON events ((payload->>'user_type'));
-- Query: WHERE payload->>'user_type' = 'admin'

-- Computed column
CREATE INDEX idx_full_name ON contacts (CONCAT(first_name, ' ', last_name));

-- Cast type
CREATE INDEX idx_created_date ON events (CAST(created_at AS DATE));
-- Query: WHERE CAST(created_at AS DATE) = '2024-01-15'

```

Index Maintenance

```

-- Check index usage
SELECT indexrelname, idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
ORDER BY idx_scan ASC; -- low idx_scan = potentially unused

-- Find unused indexes
SELECT i.indexrelname, i.tablename
FROM pg_stat_user_indexes i
WHERE i.idx_scan = 0
      AND NOT EXISTS (SELECT 1 FROM pg_constraint c WHERE c.conname = i.indexrelname);

-- Index size
SELECT pg_size_pretty(pg_relation_size('idx_name'));

-- Rebuild index (non-blocking)
REINDEX INDEX CONCURRENTLY idx_name;
REINDEX TABLE CONCURRENTLY table_name;

-- Create index without locking table
CREATE INDEX CONCURRENTLY idx_name ON t (col); -- slow, but doesn't block
reads/writes

-- Drop index without locking
DROP INDEX CONCURRENTLY idx_name;

```

When NOT to Index

- Columns with < 5 distinct values (gender, boolean flags) — seq scan is faster
- Small tables (< 1000 rows) — seq scan wins
- Columns never used in WHERE, JOIN, ORDER BY, or GROUP BY

- Very high-write, low-read tables — index overhead hurts write throughput
 - Temp tables in a single-use query
-

Quick Reference Card

Use Case	Index
Lookup by primary key	B-tree (automatic)
Email/username lookup	B-tree UNIQUE
Date range queries	B-tree
Latest records per user	B-tree composite (user_id, created_at DESC)
JSONB field search	GIN or B-tree on (col->>'field')
Array contains	GIN
Full-text search	GIN on tsvector
LIKE '%pattern%'	GIN + pg_trgm
LIKE 'prefix%'	B-tree
Geospatial proximity	GiST
No double-booking	GiST EXCLUDE
Huge append-only table, time filter	BRIN
Exact hash lookup only	Hash
Only active subset	Partial B-tree
Covering (no heap access)	B-tree with INCLUDE